UPPR: Universal Privacy-Preserving Revocation

Leandro Rometsch*, Philipp-Florens Lehwalder†, Anh-Tu Hoang*, Dominik Kaaser*, Stefan Schulte*
*Christian Doppler Laboratory for Blockchain Technologies for the Internet of Things
Institute for Data Engineering, Hamburg University of Technology, Hamburg, Germany
{leandro.rometsch, tu.hoang, dominik.kaaser, stefan.schulte}@tuhh.de
† Chair of Applied Cryptography, Technical University of Darmstadt, Darmstadt, Germany
philipp-florens.lehwalder@tu-darmstadt.de

***Abstract*—Self-Sovereign Identity (SSI) frameworks enable individuals to receive and present digital credentials in a user-controlled way. Revocation mechanisms ensure that invalid or withdrawn credentials cannot be misused. These revocation mechanisms must be scalable (e.g., at national scale) and preserve core SSI principles such as privacy, user control, and interoperability. Achieving both is hard, and finding a suitable trade-off remains a key challenge in SSI research.**

This paper introduces UPPR, a revocation mechanism for One-Show Verifiable Credentials (oVCs) and unlinkable Anonymous Credentials (ACs). Revocations are managed using per-credential Verifiable Random Function (VRF) tokens, which are published in a Bloom filter cascade on a blockchain. Holders prove non-revocation via a VRF proof for oVCs or a single Zero-Knowledge Proof for ACs. The construction prevents revocation status tracking, allows holders to stay offline, and hides issuer revocation behavior. We analyze the privacy properties of UPPR and provide a prototype implementation on Ethereum. Our implementation enables off-chain verification at no cost. On-chain checks cost 0.56–0.84 USD, while issuers pay only 0.00002–0.00005 USD per credential to refresh the revocation state.

***Index Terms*—Anonymous Credentials, Verifiable Credentials, Revocation, Privacy.**

I. Introduction

Self-Sovereign Identity (SSI) frameworks allow individuals to own and control their digital credentials without relying on centralized providers. To support this shift toward trust minimization, many SSI architectures incorporate blockchain technology as a public, tamper-resistant infrastructure. These approaches have gained momentum through initiatives such as the European Blockchain Services Infrastructure (EBSI) and the forthcoming European Digital Identity Wallet (EUDI Wallet). Typically, such SSI frameworks are structured around *issuers*, *holders*, and *verifiers*. For example, a government agency issues a digital driver's license, the holder stores it in their wallet, and later decides when and to whom to present it, such as a rental car company requesting verification. These digital attestations are referred to as Verifiable Credentials (VCs).

A key distinction for VC types is their *linkability*, that is, whether repeated presentations can be linked to the same credential instance. One-Show Verifiable Credentials (oVCs) are linkable across presentations due to static cryptographic structures, whereas Multi-Show VCs, commonly referred to as Anonymous Credentials (ACs) [5], enable unlinkable reuse through Zero-Knowledge Proofs (ZKPs). Choosing between the two depends on application-specific requirements. oVCs are efficient and supported by Trusted Platform Modules (TPMs), contributing to their adoption in the EUDI Wallet. In contrast, ACs are resource-intensive but crucial for privacy-critical scenarios such as healthcare, where usage correlation must be avoided.

One major challenge in SSI is revocation, i.e., ensuring that invalid or withdrawn credentials (e.g., a suspended license) can no longer be used. Many existing mechanisms conflict with the privacy goals of SSI [7]. In particular, they may expose the issuer's revocation behavior or allow verifiers to track holders over time based on revocation-related metadata. For instance, a verifier that once validated a driver's license should neither be able to query its status again nor infer subsequent usage from revocation-related data. Current designs often fall short of these goals, as some enable persistent revocation status monitoring by verifiers [9, 12, 13, 17, 20], while others require holders to repeatedly contact external services to keep their credentials valid [17, 18], which may be infeasible for offline or resource-constrained environments. Moreover, there are only a few approaches that provide support for unlinkable presentations as required by ACs [14].

In this work, we address these challenges by introducing *UPPR*, a *universal privacy-preserving revocation* mechanism that is compatible with both oVCs and ACs, and can be deployed on blockchain-based infrastructures. Our approach ensures *issuer privacy*, meaning no metadata of the issuer's revocation behavior (e.g., rate, timing) is revealed. Further, it preserves *holder privacy* by enabling non-revocation proofs without requiring any interaction with the issuer, a third party, or a public data source. Verifiers learn only the current revocation status at the time of presentation, without gaining information about past or future states or the ability to link multiple presentations based on revocation-related data. These properties make UPPR particularly suitable for settings with constrained credential holders, such as IoT devices, where network connectivity may be limited or unavailable.

UPPR is constructed using Verifiable Random Functions (VRFs) [8] and Bloom filter cascades [2, 21]. VRFs are the

public-key equivalent of a keyed hash function where only the holder of the private key can generate an output, while anyone with the corresponding public key can verify its correctness. Bloom filters are compact data structures that support efficient set membership checks with some probability of false positives. Bloom filter cascades eliminate false positives entirely by combining multiple filters in a layered construction.

At a high level, UPPR uses VRFs to generate *revocation tokens* that are uniquely derived from a credential but can only be linked upon presentation. If a credential is revoked, the issuer publishes the corresponding token in a public revocation artifact, such as on a blockchain. To enable efficient verification, Bloom filter cascades are used to store and test inclusion of these tokens. During presentation, the holder proves that the credential corresponds to a specific revocation token. The verifier then checks that the token is not contained in the public artifact. For oVCs, this check is performed in clear and for ACs, it is carried out using ZKPs.

Our Contribution

We summarize our individual contributions as follows:

- We propose a threat model for revocation in SSI that includes privacy risks from verifiers tracking revocation status without the holders consent, closing gaps left by previous work. Based on this model, we derive a set of concrete requirements for privacy-preserving revocation.
- We provide an overview of existing revocation mechanisms and show that they do not fulfill all requirements derived from our threat model.
- We present UPPR, a novel revocation mechanism based on VRFs and Bloom filter cascades. UPPR is universally applicable to oVCs and ACs schemes and can be deployed on blockchain-based infrastructures.
- We analyze UPPR against the derived privacy requirements and show that it satisfies all of them, making it privacy-preserving under our threat model.
- We demonstrate the practical feasibility of UPPR with an on-chain prototype and evaluate its cost and performance for both oVCs and ACs.

II. Related Work

The EBSI proposes a revocation approach based on Bloom filters [9]. However, this approach suffers from the false positive rate of standard Bloom filters and, more critically, does not allow verifiers to validate the token provided by the holder. As a result, a malicious holder could forge arbitrary tokens until one passes the non-revocation check, that is, until the computed hash appears in the Bloom filter.

Bitstring Status Lists [20] represent a simple revocation mechanism that maps credential identifiers to a public bitstring. While efficient and non-interactive from the holder's perspective, this approach leaks metadata and fails to protect the privacy of both issuers and holders.

Hyperledger AnonCreds [14] relies on cryptographic accumulators and ZKPs, making it compatible with ACs. However, the public revocation artifact reveals data about individual credentials, and holders must retrieve update information before each presentation. This frequent interaction introduces privacy and observability concerns and poses scalability challenges.

Larisch et al. [15] were the first to introduce Bloom filter cascades in the context of revocation, proposing the technique to eliminate false positives. Their work focuses on efficient revocation of TLS certificates and thus does not address comparable privacy considerations. The approach is currently used by Mozilla in their WebPKI infrastructure.

Hoops et al. [13] propose a Bloom filter cascade-based revocation mechanism for oVCs in a blockchain context. Their design achieves some privacy guarantees by salting static revocation tokens to obscure the link between a credential and its revocation status. However, revocation checks are performed solely by the verifier using a revocation token embedded in the credential. This also allows verifiers to monitor the revocation status of previously verified credentials.

Sitouah et al. [18] present a revocation mechanism based on Merkle tree accumulators that aims to prevent traceability of revocation status. However, their scheme requires a constant public credential identifier, which makes it incompatible with AC schemes. Additionally, the holder must interact with a data source to refresh their non-revocation proof, potentially leaking metadata about credential usage.

Manimaran et al. [17] introduce a revocation mechanism that combines Bloom filters with Merkle tree accumulators to improve scalability. Their work incorporates a threat model that considers issuer and holder privacy. Nonetheless, the authors acknowledge that their approach still allows verifiers to track the revocation status of previously presented credentials.

The related work reveals gaps in three main areas: (i) no protection of issuer and holder privacy, especially regarding the observability of the revocation status by verifiers; (ii) designs that require holders to fetch public revocation data prior to presentation; and (iii) reliance on static identifiers, which hinders compatibility with ACs.

To address these gaps, we introduce a threat model that captures key privacy risks in SSI revocation (Section IV). Based on this model, we derive privacy requirements (Table I), which we map to existing approaches (Table II). We then present UPPR, a revocation mechanism that addresses the privacy shortcomings of prior designs (Section V). Unlike accumulator-based schemes, which require holders to fetch the current accumulator and construct membership proofs, UPPR decouples proof generation from revocation status. Holders only attest to the correctness of their revocation token via a VRF, while verifiers determine (non-)inclusion independently using a Bloom filter cascade. The construction supports both oVCs and ACs, provides strong privacy guarantees (Section V-D), and avoids holder-side state updates, thereby enabling lightweight verification.

III. Background

In the following, we introduce the necessary background for our construction. We start with a brief overview of VCs

and their types, followed by definitions of the other primitives used in our construction.

A. Verifiable Credentials

VCs are digital assertions made by an issuer about a subject, where the authenticity of the credential can be verified cryptographically by any verifier. A typical VC contains one or more claims, a cryptographic signature from the issuer, and metadata required for verification.

There are various types of VCs, offering different levels of privacy. A key distinction is whether the credential is one-show or multi-show with respect to linkability. In a oVC, some public identifier (e.g., a signature) is revealed during each presentation, allowing verifiers to recognize multiple uses of the same credential. This linkability can compromise the holders privacy by exposing usage patterns, but may be desired in some use cases due to efficiency advantages and compatibility with regulated signature schemes and hardware support. A prominent example of an oVC is the credential format used in the EUDI Wallet.

In contrast, multi-show VCs typically rely on ZKPs to hide any identifying information, ensuring that repeated use remains unlinkable. These credentials are commonly referred to as ACs in the literature [5, 6].

We now define a minimal VC scheme suitable for constructing our revocation mechanism for oVCs and ACs later on. This definition intentionally omits multiple attributes and selective disclosure, as these can be added in a straightforward manner without affecting the validity of our construction.

Definition 1 (Verifiable Credential Scheme, cf. [4])**.** *A verifiable credential scheme* VC *consists of four algorithms:*

- $(\mathsf{ipk}, \mathsf{isk}) \xleftarrow{\$} \mathsf{VC.KeyGen}(\lambda)$*: The key generation algorithm takes a security parameter* λ *and outputs the public / secret key of the issuer* $(\mathsf{ipk}, \mathsf{isk})$.
- $\mathsf{vc} \xleftarrow{\$} \mathsf{VC.Issue}(\mathsf{isk}, \mathsf{attr})$*: The issuing algorithm takes the issuer secret key and an attribute, and outputs a credential* vc *containing the given attribute.*
- $\mathsf{pt} \xleftarrow{\$} \mathsf{VC.Present}(\mathsf{vc}, \mathsf{attr})$*: The presentation algorithm takes a credential and the attribute and outputs a presentation* pt.
- $0/1 \leftarrow \mathsf{VC.Verify}(\mathsf{ipk}, \mathsf{attr}, \mathsf{pt})$*: The verification algorithm takes the issuer public key, an attribute, and a presentation, and returns 1 if valid or 0 otherwise.*

Any VC scheme must satisfy *correctness* and *unforgeability*. Correctness ensures that credentials issued honestly by the issuer can always be successfully verified. Unforgeability means that it is infeasible to create a valid credential without access to the issuer's secret key. In the case of ACs, multiple presentations of the same credential must remain unlinkable. To support this, the verification of the attribute is not performed on a revealed value but instead through a ZKP of knowledge of a valid attribute–presentation pair. For a complete formal definition of VCs, we refer to [4].

B. Verifiable Random Functions

VRFs are cryptographic primitives that combine the properties of pseudorandom functions with public verifiability. Intuitively, a VRF allows one to generate a deterministic, pseudorandom output along with a proof that this output was correctly computed. Only the holder of the secret key can generate the output, while anyone with the corresponding public key can verify its correctness. Our construction relies on VRFs to derive unlinkable revocation tokens. For this purpose, we use the following definition.

Definition 2 (Verifiable Random Function, cf. [8])**.** *A Verifiable Random Function* VRF *consists of three algorithms:*

- $(\mathsf{pk}, \mathsf{sk}) \xleftarrow{\$} \mathsf{VRF.KeyGen}(\lambda)$*: The key generation algorithm takes as input a security parameter* λ *and outputs a public / secret key* $(\mathsf{pk}, \mathsf{sk})$.
- $(\epsilon, \rho) \leftarrow \mathsf{VRF.Eval}(\mathsf{sk}, \mathsf{m})$*: The evaluation algorithm takes as input a secret key and a message* m *and outputs a pseudorandom output* ϵ *and a proof* ρ*.*
- $0/1 \leftarrow \mathsf{VRF.Verify}(\mathsf{pk}, \mathsf{m}, \epsilon, \rho)$*: The verification algorithm takes as input a public key, a message, an output* ϵ*, and a proof* ρ*. It outputs* 1 *if the proof is valid or* 0 *otherwise.*

We require that a VRF satisfies *correctness*, *uniqueness*, and *pseudorandomness*. Correctness guarantees that any output and proof pair generated by the evaluation algorithm using a valid secret key will be accepted by the verification algorithm under the corresponding public key. Uniqueness ensures that, for any given input, it is infeasible to produce two distinct outputs that both verify successfully. Pseudorandomness guarantees that the output is computationally indistinguishable from a uniformly random string of the same length, to anyone not holding the secret key. For formal definitions of these properties, we refer to [8].

C. Bloom Filter Cascades

Bloom filters [2] are compact data structures for representing sets. They enable efficient membership testing with a configurable false positive probability, while guaranteeing no false negatives. More formally, a Bloom filter represents a set R using a bit array of length m and k independent hash functions. Elements are added by setting the bits at positions determined by the hash functions, and membership is tested by checking if all corresponding bits are set. False positives arise because multiple distinct elements may map to overlapping bit positions, causing non-members to appear present.

In our setting, we assume that the full input domain S, with $R \subseteq S$, is known in advance. This allows us to identify all false positives after the filter is constructed. Furthermore, we fix the filter's capacity to the size of the represented set $|R|$. We incorporate these assumptions into the following definition.

Definition 3 (Bloom Filter, cf. [2])**.** *A Bloom filter* BF *consists of two deterministic algorithms, adapted to our setting:*

- $(\mathbf{bf}, F) \leftarrow \mathsf{BF.Gen}(S, R, p)$*: Takes as input a domain* S*, a set* $R \subseteq S$*, and a target false positive probability*

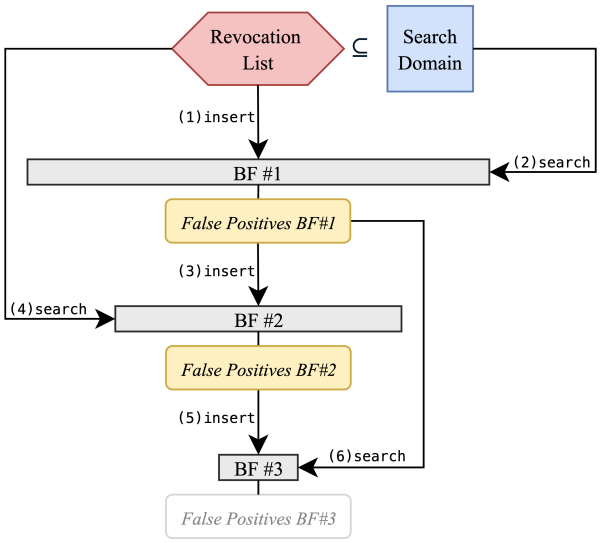

Fig. 1. Generation of a three-layer Bloom Filter (BF) cascade.

p. It outputs a m bit Bloom filter $\mathbf{bf}$ *and a set of false positives* $F \subseteq S \setminus R$.

- $b \leftarrow \mathsf{BF.Verify}(\mathbf{bf}, e)$*: Takes a Bloom filter* $\mathbf{bf}$ *and an element* $e \in S$*, and returns* $b = 1$ *if* $e \in R$*, and otherwise returns* $b = 1$ *with probability* p *or* $b = 0$.

To eliminate false positives, we make use of Bloom filter cascades [21]. The core idea is to use layers of filters, each capturing the false positives of the previous one. This approach relies on the assumption that the search domain S is known, as the false positives need to be identified during generation.

We illustrate an exemplary three-layer Bloom filter cascade in Figure 1. The first filter is constructed over a target set R, such as a revocation list, and is then queried against all elements in the domain S to identify false positives. These false positives are inserted into a second filter, which is again queried to identify its own false positives. The process continues recursively, forming a cascade of filters, until no false positives remain in the final layer.

As shown in [15], Bloom filter cascades can be constructed efficiently even for large sets, provided that the parameters are chosen appropriately. This makes them particularly suitable in our context, as they allow for a compact and false positive free representation of revocation lists while maintaining efficient membership testing.

D. Non-Interactive Zero-Knowledge Proofs

ZKPs enable a prover to convince a verifier of the truth of a statement without revealing any additional information. In the Non-Interactive Zero-Knowledge Proof (NIZK) setting, this is achieved non-interactively [3].

A NIZK is defined over a relation $\mathcal{R}$ that links public statements to private witnesses, such as proving knowledge of a preimage for a given hash. The resulting proofs are typically short and efficient to verify, making NIZKs suitable for public or repeated verification in constrained environments.

Definition 4 (Non-Interactive Zero-Knowledge Proof System, cf. [11])**.** *For our purposes, a* NIZK *proof system is built for some relation* $\mathcal{R}_{rel}$ *and consists of three basic algorithms:*

- $\mathsf{pp} \xleftarrow{\$} \mathsf{NIZK}_{rel}.\mathsf{Setup}(\lambda)$*: Generates public parameters* pp *using a security parameter* λ.
- $\pi \leftarrow \mathsf{NIZK}_{rel}.\mathsf{Prove}(\mathsf{pp}, \phi, \omega)$*: Creates a proof* π *that a public statement* ϕ *is true using a secret witness* ω.
- $0/1 \leftarrow \mathsf{NIZK}_{rel}.\mathsf{Verify}(\mathsf{pp}, \phi, \pi)$*: Checks whether a given proof is valid for the public statement.*

Such proof systems must satisfy *completeness* (valid statements can be proven), *soundness* (false statements cannot), and *zero-knowledge* (no additional information is leaked). For a formal treatment, see [11].

IV. Threat Model

In the following, we present a threat model for the revocation of VCs, from which we derive six privacy requirements summarized in Table I. As discussed in Section II, existing work typically addresses only isolated aspects of revocation privacy. We map our privacy requirements to the related work in Table II and argue about our scheme in Section V-D.

A. Adversary Capabilities

We consider a setting where VCs are issued by an *issuer* to a *holder*, and a revocation artifact is published on a publicly observable medium, such as a blockchain. This artifact contains the data needed by a *verifier* to check the revocation status of a VC. We assume the artifact is *secure*, meaning that only the issuer can produce updates and no other party can alter its content. Any external entity that receives a VC presentation acts as a verifier, responsible for checking the credential's validity and its revocation status based on the artifact. In this setting, we consider the following adversarial roles:

- **Issuer**: The issuer is assumed to be *honest-but-curious*. It correctly follows the issuance and revocation protocols but may attempt to infer any additional information about holders, such as credential usage patterns, e.g., when, how often, and where a credential is used.
- **Holder**: Holders are considered *untrusted*. While they aim to prevent any party other than the issuer from learning the revocation status of their credentials without explicit disclosure, they may also attempt to present revoked credentials to gain unauthorized access.
- **Verifier**: Verifiers are considered *untrusted*. They may attempt to infer revocation status or related metadata beyond what is explicitly disclosed during credential presentation, including efforts to link multiple presentations.
- **Third Party**: Third parties are considered *untrusted* external observers. With access to the public revocation artifact, they may monitor updates to infer which credentials have been revoked, observe revocation rates, or correlate revocation events with credential usage.

Such adversarial behavior may arise in real deployments. Consider a digital driver's license issued as a VC and used to rent vehicles. The issuer, such as a licensing authority, may

TABLE I
REQUIREMENTS FOR PRIVACY-PRESERVING REVOCATION IN SSI

ID	Name	Description
RE0	Security of Revocation	If a credential is not revoked, the holder must be able to prove its validity to any verifier (*correctness*). Moreover, it must be infeasible for any adversary to falsely prove the validity of a revoked credential (*soundness*).
RE1	Issuer Revocation Privacy	When the Issuer is revoking a credential and updating the public revocation artifact, it must not be possible for anybody to identify the revoked credential, except for the holder of that credential.
RE2	Issuer Revocation Rate	Issuers must not leak the rate at which credentials are revoked. In particular, it must be infeasible to infer the number or frequency of revocations by analyzing changes to the public revocation artifact. This also implies the timing of individual revocations must not be leaked.
RE3	Holder Non-Interaction	The holder must be able to maintain and update their revocation-related witness independently, without requiring interaction with any other party or external data source. This ensures that no entity, other than the verifier during credential presentation, can observe or infer the usage of a credential.
RE4	Holder Sovereignty	The holder must retain full control over disclosure of their revocation status; it must not be possible for a verifier to extract revocation information without the holder's active consent.
RE5	Holder Status Unlinkability	Verifying the revocation status of a credential must not reveal information about its revocation status at any previous point in time, nor enable linking or correlating multiple presentations based on revocation-related data.

TABLE II
COMPARISON TO RELATED WORK

Related Schemes	oVC	AC	RE0	RE1	RE2	RE3	RE4	RE5
Bitstring Status List [20]	✓	✗	✓	✗	✗	✓	✗	✗
AnonCreds Rev. [14]	✗	✓	✓	✗	✗	✗	✓	✓
CRSet [13]	✓	✗	✓	✓	✓	✓	✗	✗
Prevoke [17]	✓	✗	✓	✓	✓	✗	✓	✗
Sitouah et al. [18]	✓	✗	✓	✓	✓	✗	✓	✓
UPPR (Our Work)	✓	✓	✓	✓	✓	✓	✓	✓

attempt to unlawfully monitor when and where credentials are presented. A holder whose license has been revoked may still try to rent a car using outdated credential data. Verifiers, such as rental companies, may track repeated visits to build customer profiles. Third parties observing the revocation artifact may infer broader patterns, such as spikes in revocations indicating enforcement actions. These behaviors must be mitigated to uphold the principle of user control that underpins SSI. Accounting for these attack vectors is therefore essential in the design of privacy-preserving revocation mechanisms.

B. Privacy Requirements

We define a set of privacy requirements that directly address the adversarial capabilities described in Section IV-A. These are summarized in Table I. While primarily focused on privacy, requirement RE0 ensures baseline security by guaranteeing that only non-revoked credentials can be successfully presented. This prevents untrusted holders from using revoked credentials to gain unauthorized access.

To mitigate the threat posed by honest-but-curious issuers, RE3 is employed. It ensures that holders can maintain their revocation-related data (e.g., proof or token) independently, without interacting with the issuer or any other party. This prevents the observation of credential usage and inferring behavioral patterns over time.

To address the capabilities of untrusted verifiers, requirements RE3, RE4, and RE5 jointly mitigate the risk of revocation-based inference and linking. RE3 prevents indirect leakage of usage information through interaction during witness updates. RE4 enforces that revocation status is disclosed only with the holder's explicit consent, preventing unsolicited queries. RE5 ensures unlinkability across presentations by preventing correlation of revocation-related data over time. These safeguards collectively protect against verifier-driven profiling, status probing, and cross-session linkage.

Untrusted third parties may passively monitor the public revocation artifact to infer credential status or issuer behavior. Requirements RE1 and RE2 mitigate such risks. RE1 ensures that artifact updates do not reveal which credential was revoked, protecting against targeted identification. RE2 conceals the timing and frequency of revocations, blocking statistical analysis that could expose systemic behavior or operational trends. Together, these requirements defend against observational inference attacks on the revocation infrastructure.

Altogether, these requirements provide targeted mitigation for the threats identified in our model. While agnostic to specific implementations, they offer a foundation for designing revocation mechanisms that uphold SSI principles.

V. CONSTRUCTION OF UPPR

In this section, we present UPPR. It supports both oVCs and ACs, and can be deployed on blockchain-based or traditional infrastructure. As shown in Section V-D, UPPR satisfies all requirements defined in Section IV-B and is therefore privacy-preserving under our threat model.

The core idea of UPPR is to bind a VRFs to each credential via a VC attribute containing the VRF public key, with the secret key shared between issuer and holder. Revocation is realized through a blocklist: the issuer publishes the VRF output (*revocation token*) of revoked credentials to a public artifact. Only the holder can prove non-revocation by providing its own revocation token and a VRF proof. Upon presentation, the verifier validates the token via the VRF proof and checks the token's non-inclusion in the artifact, confirming the credential is not revoked.

To support scalable on-chain verification, we adopt a technique introduced by Larisch et al. [15], storing revocation tokens in a cascade of Bloom filters. Unlike with cryptographic accumulators used in other works [14, 17, 18], this design avoids the need for the holder to retrieve the current revocation artifact, as the VRF proof is generated independently of it.

In the following, we present the issuance procedure, followed by the revocation and verification mechanism. Finally, we analyze the security and privacy of our construction.

A. Issuance

Any VC scheme adopting our revocation mechanism must extend its issuance procedure by generating a VRF key pair for each VC. The VRF public key is then embedded as an attribute. For simplicity, we assume that the VRF public key is the sole attribute of the credential. More formally, the procedure is as follows:

Given an issuer with key pair $(\mathsf{ipk}, \mathsf{isk})$, the issuer generates a fresh VRF key pair $(\mathsf{pk}, \mathsf{sk}) \xleftarrow{\$} \mathsf{VRF.KeyGen}(\lambda)$ for each new credential. The credential is then created as $\mathsf{vc} \xleftarrow{\$} \mathsf{VC.Issue}(\mathsf{isk}, \mathsf{pk})$. The resulting credential vc and its corresponding VRF secret key sk are securely transmitted to the holder. The issuer retains the pair $(\mathsf{vc}, \mathsf{sk})$ in a local set S, referred to as the *domain* of all issued credentials.

B. Revocation

We now present the revocation mechanism given in Algorithm 1. On a high-level it consists of four steps: (1) the issuer computes the VRF evaluations of all revoked credentials for the current epoch, creating a set of revocation tokens; (2) this set is padded with random values to hide the true number of revocations; (3) the issuer computes a set of VRF outputs for all non-revoked credentials to test against false positives; and (4) a cascade of Bloom filters is constructed based on the revocation token set, tested against the non-revoked credential token set. In Figure 1 we illustrate this procedure for a three-layer Bloom filter cascade. This cascade acts as the revocation artifact, which can then be published on-chain or in a centralized database. For the input and output of Algorithm 1 we introduce the following notation:

Let λ be the security parameter. Let R_{sk} and S^*_{sk} be the VRF secret keys of revoked and non-revoked credentials, respectively. Let $c_{\max}$ be the upper bound on the number of revocations per epoch, p the desired false positive rate of a single Bloom filter, and t the current epoch. The algorithm outputs a n-layer cascade of Bloom filters $BF[0 \dots n-1]$ which serves as the revocation artifact for epoch t.

Algorithm 1: Issuer Revocation

Input: $\lambda, R_{\mathsf{sk}}, S^*_{\mathsf{sk}}, c_{\max}, \mathsf{t}, p$
Output: Cascade of n Bloom filters $BF[0...n-1]$

```
initialize R_rt ← ∅, S*_rt ← ∅;
foreach sk ∈ R_sk do // Step 1: Gen revocation tokens
    (rt, _) ← VRF.Eval(sk, t);
    R_rt ← R_rt ∪ {rt};
for i ← 1 to c_max − |R_rt| do // Step 2: Add padding
    rt$ ←$ {0,1}^λ;
    R_rt ← R_rt ∪ {rt$};
foreach sk ∈ S*_sk do // Step 3: Gen false positive test set
    (rt*, _) ← VRF.Eval(sk, t);
    S*_rt ← S*_rt ∪ {rt*};
BF ← [], FP ← [], bf_input ← R_rt, depth ← 0;
repeat // Step 4: Gen Bloom filter cascade
    switch depth do
        case 0 do
            dom ← S*_rt;
        case 1 do
            dom ← R_rt;
        otherwise do
            dom ← FP[depth − 2];
    BF[depth], FP[depth] ← BF.Gen(dom, bf_input, p);
    bf_input ← FP[depth];
    depth ← depth + 1;
until |FP[depth − 1]| == 0;
return BF;
```

C. Verification

We now present the verification algorithm, which differs between oVC and AC schemes. The central distinction is the method used to establish the validity of the revocation token during credential presentation. As outlined in Section III-A, oVC schemes reveal a static identifier upon each use. Since unlinkability is not required in this setting, the holder may directly disclose the VRF public key to the verifier. This enables the verifier to validate the revocation token in the clear using the accompanying VRF proof. However, the verification process must still ensure that no revocation status can be inferred across different epochs. In contrast, AC schemes preserve unlinkability. Consequently, the VRF public key of the AC must remain hidden. Hence, the holder proves in zero-knowledge that the revocation token was correctly generated. Despite this distinction, both schemes share the same verification procedure. Once the verifier validated the revocation token, it checks that the token is not contained in the Bloom filter cascade. This non-inclusion check serves as confirmation that the credential has not been revoked.

In the following, we describe the procedures used to establish revocation token validity. We then present the algorithm for verifying non-inclusion in the Bloom filter cascade.

1) One-show Credentials: For oVCs, the verification procedure is divided into two phases. First, the holder generates a presentation of the credential, which includes the VRF public key as an attribute. The holder then computes the revocation token and the corresponding VRF proof for the current epoch. The verifier subsequently validates the presentation and checks the correctness of the revocation token using the disclosed VRF public key. More formally, the procedure is as follows:
Holder. The holder computes a presentation pt of the credential vc via $\mathsf{pt} \leftarrow \mathsf{VC.Present}(\mathsf{vc}, \mathsf{pk})$, where pk denotes the VRF public key embedded in the credential. The holder then computes the revocation token rt and its associated proof ρ

for the current epoch t by invoking $(\mathsf{rt}, \rho) \leftarrow \mathsf{VRF.Eval}(\mathsf{sk}, \mathsf{t})$, where sk is the VRF secret key corresponding to pk. The holder transmits the tuple $(\mathsf{pt}, \mathsf{pk}, \mathsf{rt}, \rho)$ to the verifier.

Verifier. Upon receiving $(\mathsf{pt}, \mathsf{pk}, \mathsf{rt}, \rho)$, the verifier first validates the correctness of the presentation using the issuers public key ipk by checking $1 = \mathsf{VC.Verify}(\mathsf{ipk}, \mathsf{pk}, \mathsf{pt})$. It then verifies the revocation token by evaluating $1 = \mathsf{VRF.Verify}(\mathsf{pk}, \mathsf{t}, \mathsf{rt}, \rho)$. If both succeeds, the verifier knows that rt is the revocation token for credential vc at epoch t.

2) Multi-show Credentials: ACs inherently rely on ZKPs for credential presentation. To support revocation in such schemes under our approach, we extend the presentation protocol with an additional constraint. This constraint proves in zero-knowledge, that the revocation token was correctly derived using the VRF public key as a hidden witness.

To formalize this, we define a new relation $\mathcal{R}_{\mathsf{pres}}$. The public input to the relation consists of the issuers public key ipk, the revocation token rt, and the epoch t. The private input includes the presentation pt, the VRF public key pk, and the corresponding VRF proof ρ. The relation $\mathcal{R}_{\mathsf{pres}}$ ensures that the prover possesses a valid credential and that the revocation token rt is correctly derived for that credential at epoch t

$$\begin{aligned}\mathcal{R}_{\mathsf{pres}} := \big\{(\phi_{\mathsf{pres}} = (\mathsf{ipk}, \mathsf{rt}, \mathsf{t}),\ \omega_{\mathsf{pres}} = (\mathsf{pt}, \mathsf{pk}, \rho)) \ \big|\ \\ 1 = \mathsf{VC.Verify}(\mathsf{ipk}, \mathsf{pk}, \mathsf{pt}) \wedge \\ 1 = \mathsf{VRF.Verify}(\mathsf{pk}, \mathsf{t}, \mathsf{rt}, \rho)\big\}\end{aligned} \tag{1}$$

We assume a NIZK proof system $\mathsf{NIZK}_{\mathsf{pres}}$ for the relation $\mathcal{R}_{\mathsf{pres}}$, instantiated with public parameters pp. The verification procedure is structured into a local computation step by the holder and a subsequent verification step by the verifier.

Holder. The holder, in possession of a credential vc, locally computes the tuple $(\mathsf{pt}, \mathsf{pk}, \mathsf{rt}, \rho)$ for the current epoch t as described in Section V-C1. A ZKP π is then generated by invoking $\pi \leftarrow \mathsf{NIZK}_{\mathsf{pres}}.\mathsf{Prove}(\mathsf{pp}, \phi_{\mathsf{pres}}, \omega_{\mathsf{pres}})$, where $\phi_{\mathsf{pres}} = (\mathsf{ipk}, \mathsf{rt}, \mathsf{t})$ and $\omega_{\mathsf{pres}} = (\mathsf{pt}, \mathsf{pk}, \rho)$. The holder transmits the tuple (rt, π) to the verifier.

Verifier. Upon receiving (rt, π), the verifier checks the validity of the proof by evaluating $\mathsf{NIZK}_{\mathsf{pres}}.\mathsf{Verify}(\mathsf{pp}, \phi_{\mathsf{pres}}, \pi)$. If the verification succeeds, the verifier is convinced that rt is the correctly derived revocation token for the credential vc in epoch t, without learning any linkable information.

3) Non-Inclusion Check of Revocation Token: To verify that a revocation token is not contained in the Bloom filter cascade, and thus that the corresponding credential has not been revoked, we invoke Algorithm 2. The algorithm takes as input the n-layer cascade $BF[0 \dots n{-}1]$ and a revocation token rt. Recall from Section III-C that even-indexed layers encode true positives, while odd-indexed layers contain the false positives of the preceding layer. The algorithm sequentially evaluates each layer for a match against rt.

If any odd-indexed layer does not contain the token, this implies that the token was a false positive in a previous even layer. In this case, the algorithm returns 1, indicating that the token is not revoked. If the token is still matched in the final layer, revocation is decided based on the parity of the layer: it is considered revoked if the final matching layer is even, and not revoked if it is odd.

Algorithm 2: Revocation Status Check

Input: Cascade $BF[0 \dots n{-}1]$, token rt
Output: 1 if token is *not* revoked, 0 if revoked

```
for i ← 0 to n−1 do
    b ← BF.Verify(BF[i], rt);
    if b = 0 and i is odd then
        return 1 // Early accept: token not revoked
    if i = n−1 then
        return (i mod 2 = 1) // Accept, if match in odd layer
```

D. Security and Privacy

We now informally argue the security of our construction based on the properties of the underlying cryptographic primitives introduced in Section III. For the requirements of a privacy-preserving revocation system we refer to Table I.

a) Security of Revocation (RE0): To establish that RE0 is satisfied, we demonstrate that the revocation mechanism fulfills both *correctness* and *soundness*.

Correctness holds since the Bloom filter cascade only contains revocation tokens, i.e., outputs of the VRF computed over revoked credentials, as well as dummy values. These dummy values are randomly chosen based on the security parameter, such that the probability of a collision with a valid revocation token is negligible. As detailed in Section V-B, false positives are eliminated through a layered cascade construction. Consequently, a revocation token corresponding to a non-revoked credential will not be contained in the cascade, and the holder can successfully demonstrate non-revocation. Verification of the revocation token relies on the correctness of the underlying VRF scheme and the correctness of the VC scheme used for credential presentation. In the case of ACs, this verification is additionally performed in zero-knowledge, relying on this security property of the employed NIZK system.

Soundness is ensured by the uniqueness property of the VRF. For each epoch, there exists exactly one valid revocation token under the VRF key bound to the credential. The token's correctness is verified either directly, using the VRF public key embedded in the credential (in the oVC case), or via a ZKP that attests to the association between the hidden VRF key and the presented credential (in the AC case). In the latter, soundness also depends on the soundness of the underlying NIZK system. It is therefore computationally infeasible for an adversary to forge a valid revocation token that differs from the one generated by the issuer and included in the cascade. Finally, as the Bloom filter cascade is constructed to eliminate false positives and by design exhibits no false negatives, any inclusion check on a correctly inserted revocation token will yield the correct result.

b) Issuer Revocation Privacy and Rate (RE1 & RE2): Due to the pseudorandomness of the VRF, each revocation token is computationally indistinguishable from a random

value without knowledge of the corresponding VRF secret key. As a result, the public revocation artifact reveals no information about the underlying credentials. To address revocation rate privacy, the revocation artifact is padded with randomly generated dummy tokens of the same format and length. This obfuscates the actual number of revoked credentials. Consequently, even an adversary with access to the full history of revocation artifacts can at most infer an upper bound on the number of revocations, determined by the configured capacity of the Bloom filter cascade.

c) Holder Non-Interaction (RE3): The holder can compute the revocation token locally using only their credential and the current epoch, without requiring access to the public revocation artifact or any other party. Also, the proof that the revocation token was correctly generated can be computed locally in both the oVC and AC cases, as it only requires the VRF secret key. This ensures that the holder retains full control over their credential and can reveal its revocation status without involving the issuer or any other entity.

d) Holder Sovereignty (RE4): It follows from the pseudorandomness of the VRF that only the holder (and issuer) having knowledge of the VRF secret key can evaluate the VRF output for a given input. Therefore, only the holder can generate a valid revocation token for their credential for a specific epoch, which it can then present to a verifier. Notably, if the scheme supports selective disclosure, the holder can choose to disclose the VRF public key selectively during presentation. This allows the holder to retain full control over whether the verifier can check the revocation status of the credential, even if other attributes are disclosed. Naturally, the verifier may choose to reject the presentation if the revocation status is not revealed, but the decision to disclose remains entirely with the holder.

e) Holder Status Unlinkability (RE5): In the case of oVCs, unlinkability across presentations is not a goal, as the credential itself reveals a stable identifier that enables linking by design. Nonetheless, RE5 still applies in the sense that a verifier must not be able to infer revocation status at any point in time other than the current epoch. That is, even if the credential is inherently linkable, revocation-related data must not permit status tracking across epochs. In UPPR, the revocation token is specific to the current epoch and derived using a VRF, which ensures that revocation tokens differ across epochs and are pseudorandom without knowledge of the VRF secret key. As such, a verifier cannot predict past or future tokens and thus cannot infer historical or future revocation states of a known credential.

In the case of ACs, where unlinkability is a strict requirement, the verifier receives only the revocation token and a ZKP that it was correctly derived. Assuming the NIZK system satisfies perfect zero-knowledge, no information about the VRF key or credential is leaked. Each revocation token is specific to an epoch and independent of all others, preventing correlation of presentations across time. Therefore, even when the same credential is presented in multiple epochs, the verifier cannot link those presentations or deduce whether the credential was revoked in the past.

VI. Evaluation

We evaluate our open-source prototype implementation[1] of UPPR for both the oVC and AC use cases, focusing on performance and cost in a realistic deployment setting with full on-chain verification.

The backend is implemented in Go and the smart contracts in Solidity. For the VRF, we use a optimized version of `go-ecvrf`[2] in the backend and `vrf-solidity`[3] on-chain. In the AC case, we generate the necessary ZKP using `gnark`[4] compiled to a Groth16 [10] circuit. All benchmarks are performed on a 12-core Apple M4 Pro CPU with 48 GB RAM and repeated $N = 100$ times. Cost estimates assume an average gas price of 1 Gwei and an exchange rate of 2,000 USD per ETH, reflecting conditions in June 2025.

In the following, we describe the computational cost of generating revocation artifacts and tokens, followed by an analysis of the on-chain gas cost for updating the revocation artifact and verifying non-revocation of VC presentations. We conclude with a discussion of the results and their implications for practical deployments.

A. Off-Chain Computational Cost

As specified in Section V, both the issuer and the holder must generate revocation tokens. The issuer derives tokens for revoked credentials to construct the Bloom filter cascade and uses tokens from non-revoked credentials to suppress false positives. On the holder side, a revocation token must be generated for each credential presentation per epoch.

In the oVC case, the issuer derives the revocation token using a VRF, and the holder provides a corresponding VRF proof during presentation. Our implementation employs a standardized[5] signature-based VRF, where the proof is a deterministic signature over the epoch hash and verification reduces to checking the signature. In contrast, the AC case employs ZKPs, allowing us to omit the signature step. The revocation token is computed by hashing the epoch with the VRF secret key, which the holder supplies as a private input to the circuit. The proof verifies consistency with the VRF public key in the credential. As a result, UPPR can use a reduced VRF in the AC setting, relying only on a key pair and a hash function.

a) Revocation Token Generation: Table III summarizes the computational cost for token and proof generation. Simple token generation is fast in both cases: 29µs for oVC and 10µs for AC, the latter benefiting from the simplified VRF.

Concerning proof generation, in the oVC case, this is a VRF proof of 73µs, whereas in the AC case, it is a ZKP of 33.7 ms, with a peak memory usage of 5.335 MB. The ZKP generation is significantly more computationally heavy

[1] https://github.com/leandro-ro/UPPR
[2] https://github.com/vechain/go-ecvrf
[3] https://github.com/witnet/vrf-solidity
[4] https://github.com/Consensys/gnark
[5] https://datatracker.ietf.org/doc/draft-irtf-cfrg-vrf/09/

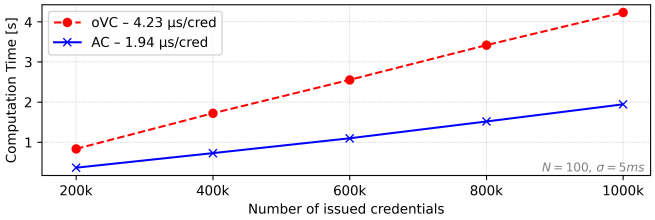

Fig. 2. Computation Time for Revocation Artifact Generation.

TABLE III
COMP. COST FOR GENERATING A REVOCATION TOKEN & PROOF.

	Token Gen.	Proof	P. Time	P. Memory
oVC	29µs (±3µs)	VRF.Eval	73µs (±8µs)	0.004MB
AC	10µs (±1.8µs)	NIZK.Prove	33.7ms (±1ms)	5.335MB

than the VRF proof, which is expected due to the complexity of ZKPs. Notably, the ZKP also includes a check for the correctness of the credential representation. This reduces on-chain cost for the AC case, as presented in Section VI-B2.

b) Revocation Artifact Generation: In each epoch, the issuer constructs a Bloom filter cascade from all revocation tokens. This process scales linearly with the number of credentials. Token generation is parallelized, while cascade construction remains sequential. To reduce cascade size, we adopt the false positive strategy by Hoops et al. [13], using an adaptively small false positive rate for the first layer and fixing all subsequent layers at 50%. As most credentials are resolved in the first layer, deeper layers are rarely accessed and contribute negligibly to the overall cost.

Figure 2 shows generation times for up to one million credentials. The oVC case averages 4.2 µs per credential, and the AC case 1.9 µs, yielding total times of 4.23 s and 1.94 s for one million credentials, respectively. This reflects the efficiency gains from the simplified VRF in the AC case.

B. On-Chain Gas Cost

We distinguish between two forms of on-chain interaction: (i) updating the revocation artifact (by the issuer), and (ii) verifying a credential presentation and revocation status (by the verifier smart contract). One-time setup costs are excluded, as they become negligible during sustained system operation.

1) Revocation Artifact Update: A new Bloom filter cascade is uploaded to the blockchain each epoch. In Ethereum, writing fresh storage costs 20,000 gas per 32 bytes, while updating existing storage is cheaper at 5,000 gas per 32 bytes. Thus, updates are expected to be one fourth of the initial cost.

Figure 3 reports gas usage and ETH cost for varying capacities and domain size. For a domain of one million credentials with a 10% revocation capacity (i.e., up to 100,000 revoked credentials), the initial upload costs approximately 0.08 ETH, while subsequent updates require around 0.02 ETH. At 5% capacity, each update costs approximately 0.0025 ETH. This corresponds to 25.80 USD and 43.20 USD per update for

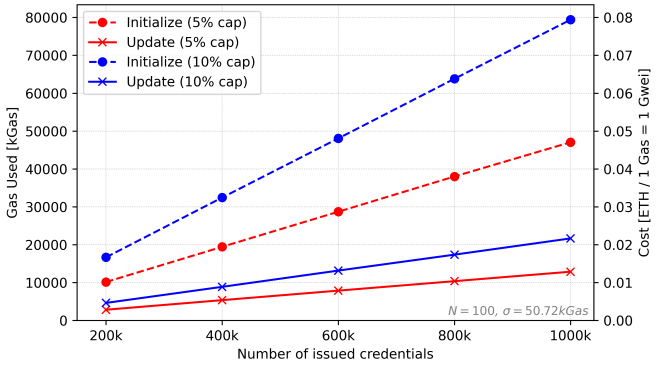

Fig. 3. Gas Cost for Updating the Revocation Artifact.

5% and 10% capacity, respectively, resulting in a per-credential update cost of roughly 0.0000258 USD and 0.0000432 USD. Notably, the cost increase with higher revocation capacity grows less than proportionally, due to the lower false positive resistance required, which slows cascade growth.

2) Credential Verification: The verification function is read-only and incurs no gas cost when called from off-chain applications. Gas is only consumed if invoked by another smart contract, such as within a decentralized application.

The verifier checks both the credential and its revocation status. In the AC case, a single ZKP proves both, while the oVC case requires separate checks for the credential and the VRF proof. The AC case incurs 281,741 gas (0.56 USD), compared to 422,209 gas (0.84 USD) for oVC.

C. Discussion

The evaluation demonstrates the feasibility of using UPPR for revocation of both linkable oVCs and unlinkable ACs. A core advantage of the mechanism are the implications of its integration with blockchain infrastructure. Revocation artifacts are stored directly on-chain, enabling off- and on-chain applications to verify revocation status without relying on issuer-hosted infrastructure. This avoids assumptions about the availability or integrity of external services and ensures long-term verifiability. Issuers only incur on-chain costs during artifact updates, which can be scheduled flexibly to reduce cost or triggered immediately when necessary.

Credential holders benefit from efficient and privacy-preserving proof generation that only requires demonstrating the validity of their revocation token. This is possible because the actual non-revocation check is decoupled and performed independently by the verifier. In the case of oVCs, proving token validity is fast and lightweight. Since only verifiers access the revocation artifact, holders can remain offline. This makes the scheme particularly suitable for constrained and latency-sensitive settings, such as the IoT.

As verifiers are responsible for checking token validity and non-revocation, they incur moderate on-chain costs. If the verifier runs off-chain, the check is free, as it only involves a on-chain read and some local computation. The AC case is more gas-efficient due to constant-time ZKP verification. In contrast,

the oVC case is limited by the current VRF implementation. Replacing it with a native or precompiled version, as used by Chainlink or Algorand [1], could significantly lower verifier costs without affecting functionality.

Moreover, our evaluation is based on Ethereum, one of the more expensive general-purpose blockchains. Some Layer 1 and Layer 2 alternatives offer significantly lower transaction fees and storage cost [16, 19]. In this context, the gas-based cost estimates reported here represent conservative upper bounds. Deployment on cheaper chains would further improve cost and extend the applicability of UPPR to broader settings.

Finally, the operational cost is borne by the issuer, who is typically a centralized authority responsible for credential lifecycle management. Per-credential update costs remain a fraction of a cent even in large-scale settings. In return, the issuer gains a scalable and resilient revocation mechanism that eliminates the need for maintaining an online verification service, thereby avoiding availability risks or central points of failure. Notably, UPPR preserves the privacy guarantees required by SSI ecosystems, and does so at a cost that remains manageable compared to other revocation systems, where privacy is often sacrificed for efficiency (cf. Table II).

VII. Conclusion

Revocation is a persistent weak point in SSI deployments. Most current mechanisms either undermine issuer privacy, burden holders with interaction, or expose them to linkage and tracking. UPPR closes this gap by combining VRFs with a Bloom filter cascade to deliver a universal protocol for both linkable oVC and unlinkable AC schemes. In UPPR, revocation artifacts do not leak metadata, holders never need to 'phone home', and verifiers learn nothing beyond the moment-in-time revocation status for the scope of the current credential presentation. Our analysis confirms conformity with all six privacy requirements introduced in our threat model, while our open-source prototype shows that generating artifacts for one million credentials completes in seconds off-chain and that on-chain costs remain subcent level per credential, even on costly networks like Ethereum.

Future work will focus on (i) cutting on-chain costs by exploiting precompiled VRF primitives offered natively on certain blockchains, and (ii) exploring constructions that allow for multiple unlinkable revocation tokens per epoch.

Acknowledgment

The financial support by the Austrian Federal Ministry of Economy, Energy and Tourism, the National Foundation for Research, Technology and Development and the Christian Doppler Research Association is gratefully acknowledged. Further, this result is part of a project that received funding from the European Research Council (ERC) under the European Union's Horizon 2020 and Horizon Europe research and innovation programs (grant CRYPTOLAYER-101044770). We also thank Mirko Mollik of the German Federal Agency for Breakthrough Innovation (SPRIND) for providing valuable industry insights.

References

[1] G. Blaut, X. Ma, and K. Wolter, "Exploring Randomness in Blockchains," in *2023 IEEE International Conference on Blockchain and Cryptocurrency, ICBC*, 2023, pp. 1–5.

[2] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[3] M. Blum, P. Feldman, and S. Micali, "Non-interactive zero-knowledge and its applications," in *Twentieth Annual ACM Symposium on Theory of Computing, STOC*, 1988, pp. 103–112.

[4] J. Camenisch, S. Krenn, A. Lehmann, G. L. Mikkelsen, G. Neven, and M. Ø. Pedersen, "Formal treatment of privacy-enhancing credential systems," in *International Conference on Selected Areas in Cryptography*, 2015, pp. 3–24.

[5] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *20th International Conference on the Theory and Applications of Cryptographic Techniques, EUROCRYPT*, 2001, pp. 93–118.

[6] D. Chaum, "Security without identification: Transaction systems to make big brother obsolete," *Communications of the ACM*, vol. 28, no. 10, pp. 1030–1044, 1985.

[7] R. Chotkan, J. Decouchant, and J. Pouwelse, " Distributed Attestation Revocation in Self-Sovereign Identity," in *IEEE 47th Conference on Local Computer Networks, LCN*, 2022, pp. 414–421.

[8] Y. Dodis and A. Yampolskiy, "A verifiable random function with short proofs and keys," in *8th International Conference on Theory and Practice in Public Key Cryptography*, 2005, pp. 416–431.

[9] *EBSI Credential Status Framework (VCS)*, https://hub.ebsi.eu/vc-framework/credential-status-framework/vcs, Accessed: 2025-06-30.

[10] J. Groth, "On the Size of Pairing-Based Non-Interactive Arguments," in *35th International Conference on the Theory and Applications of Cryptographic Techniques, EUROCRYPT*, 2016, pp. 305–326.

[11] J. Groth, R. Ostrovsky, and A. Sahai, "Perfect non-interactive zero knowledge for NP," in *25th International Conference on the Theory and Applications of Cryptographic Techniques, EUROCRYPT*, Springer, 2006, pp. 339–358.

[12] A.-T. Hoang, C. U. Ileri, W. Sanders, and S. Schulte, "zkSSI: A Zero-Knowledge-Based Self-Sovereign Identity Framework," in *2024 IEEE International Conference on Blockchain*, 2024, pp. 276–285.

[13] F. Hoops, J. Gebele, and F. Matthes, "CRSet: Non-Interactive Verifiable Credential Revocation with Metadata Privacy for Issuers and Everyone Else," *arXiv preprint arXiv:2501.17089*, 2025.

[14] Hyperledger AnonCreds Working Group, *AnonCreds Specification*, https://hyperledger.github.io/anoncreds-spec/, Accessed: 2025-06-30.

[15] J. Larisch, D. Choffnes, D. Levin, B. M. Maggs, A. Mislove, and C. Wilson, "CRLite: A Scalable System for Pushing All TLS Revocations to All Browsers," in *2017 IEEE Symposium on Security and Privacy, SP*, 2017, pp. 539–556.

[16] N. E. Majd, A. Hinojosa, C. Fisher, F. Landeros, and F. Baldimtsi, "A study on Gas Optimization and Performance Evaluation of Sui and Aptos blockchains," in *7th IEEE/ACM International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2025, pp. 9–16.

[17] P. Manimaran, A. F. Da Conceição, T. Garrett, M. Raikwar, and R. Vitenberg, "Prevoke: Privacy-Preserving Configurable Method for Revoking Verifiable Credentials," in *2024 IEEE International Conference on Blockchain*, 2024, pp. 354–361.

[18] N. Sitouah, F. Bruschi, F. L. Pallotta, R. Mencucci, and D. Sciuto, "An Untraceable Credential Revocation Approach Based on a Novel Merkle Tree Accumulator," in *2024 IEEE International Conference on Blockchain and Cryptocurrency, ICBC*, 2024, pp. 210–214.

[19] F. Stiehle and I. Weber, "The Cost of Executing Business Processes on Next-Generation Blockchains: The Case of Algorand," in *Business Process Management: Blockchain, Robotic Process Automation, Central and Eastern European, Educators and Industry Forum*, 2024, pp. 89–105.

[20] W3C, *Verifiable Credentials Bitstring Status List*, https://www.w3.org/TR/vc-bitstring-status-list/, Accessed: 2025-06-30.

[21] Z. Wang, "CasAB: Building Precise Bitmap Indices via Cascaded Bloom Filters," in *Fourth International Conference on Internet Computing for Science and Engineering*, 2009, pp. 85–92.